

Assignment-11: Concept of Pipeline

1. Learning Objectives

- Mastering the end-to-end Machine Learning pipeline flow: **Data** → **EDA** → **Split** → **Pipeline Fit** → **Pipeline Predict**.
- Understanding the importance of fitting preprocessing parameters (scaling/transformation) **only** on training data to prevent data leakage.
- Building a modular Scikit-learn Pipeline that integrates feature engineering, imputation, and modeling.
- Evaluating regression performance using standardized metrics and diagnostic plots .

2. Dataset and Target Variable

Item	Requirement
CSV file name	Diamond Price Prediction.csv
Target column	Price (Do not transform or scale the target).
Numeric features	carat, depth, table, x, y, z, volume
Transformation required	Log transformation only

3. Preferable Notebook Structure

Your notebook should follow the structure as mentioned below. Do not create unnecessary extra workflow sections.

Notebook Cell	What it should contain
Cell 0	Markdown introduction explaining the workflow and main focus on pipeline.
Cell 1	Imports, EDA helper function, dataset loading function, standardization helper, and manual log preprocessing function.
Cell 2	Pipeline helper function, metric function, diagnostic plot function, and final log pipeline function.
Cell 3	Run Part-1: initial EDA and manual preprocessing plots.
Cell 4	Run Part-2: final pipeline execution.

4. Step-by-Step Tasks

Task A — Create the EDA helper function

Create a helper function called **print_diagnostics()**. This function should print basic information and generate plots for the dataset.

- Print total duplicate rows, dataset shape, and total missing values.
- Print column-wise missing values if present.
- Create boxplots, histograms and correlation heatmap for numeric columns.
- Create frequency/count plots for categorical columns.
- Create a consolidated boxplot excluding the target column price.

Task B — Load the dataset and rename columns

Create a function called **analyze_diamond_data()**. It should read the CSV file, rename long column names to simple names, run initial EDA, and return the dataframe.

Sample code:

```
def analyze_diamond_data(file_path='Diamond_Price_Prediction.csv'):
    df = pd.read_csv(file_path)
    df = df.rename(columns={
        'Carat(Weight of Daimond)': 'carat',
        'Cut(Quality)': 'cut',
        'Color': 'color',
        'Clarity': 'clarity',
        'Depth': 'depth',
        'Table': 'table',
        'Price(in US dollars)': 'price',
        'X(length)': 'x',
        'Y(width)': 'y',
        'Z(Depth)': 'z'
    })
    print_diagnostics(df, "Initial EDA - Untransformed Data", target_col='price')
    return df
```

Task C — Create the standardization helper

Create **apply_standardization()**. This helper is for the manual preprocessing part. It should fit StandardScaler on training data only and transform test data using the same scaler. Sample code:

```
def apply_standardization(train_df, test_df, numeric_cols, target_col='price'):
    scaler = StandardScaler()
    cols_to_scale = [c for c in numeric_cols if c != target_col]
    train_df[cols_to_scale] = scaler.fit_transform(train_df[cols_to_scale])
    test_df[cols_to_scale] = scaler.transform(test_df[cols_to_scale])
    return train_df, test_df
```

Task D — Manual preprocessing for transformed plots (Note : For your own understanding of how log transformations and scaling change your data distribution.)

Create **preprocess_diamond_log()**. This part is mainly to show the effect of log transformation and scaling on training data.

- Copy the original dataframe.
- Replace zero values in x, y, and z with NaN because diamond dimensions cannot be zero.
- Create a numeric feature named volume using $x * y * z$.
- Drop cut, color, and clarity because no encoding is used.
- Split the data into train and test before transformation.
- Apply $\text{np.log1p}()$ only on independent numeric columns.
- Print diagnostics for Train (Post Log, Pre Scaling).
- Apply standardization using train data only.
- Print diagnostics for Train (Post Log & Scaling).

Task E — Create the feature engineering helper functions

Create a helper function named **feature_engineer_numeric_only()** for the pipeline. This function should prepare only the numeric input features for the model. It should copy the input data, replace invalid zero values in the diamond

dimension columns x, y, and z with missing values, create a new numeric feature called volume using $x \times y \times z$, remove the categorical columns cut, color, and clarity, and return only the numeric columns needed for modeling: carat, depth, table, x, y, z, and volume. **Do not** apply any categorical encoding in this function.

Task F — Create model metric and diagnostic functions

Create helper functions for model evaluation and diagnostic plots. The helper functions should:

1. Calculate the main regression metrics:
 - RMSE
 - MAE
 - R^2 score
2. Print the metrics clearly for both:
 - training data
 - test data
3. Create an **Actual vs Predicted** plot to compare the real diamond prices with the predicted prices.
4. Create a **Residuals vs Predicted** plot to check whether the prediction errors show any pattern.
5. Create a **QQ plot of residuals** to check whether the residuals approximately follow a normal distribution.
6. Use these helper functions after generating predictions from the pipeline.

Task G — Build and run the final log pipeline

Create a function named `run_pipeline_log()`. This is the main function of the assignment. It should take the cleaned diamond dataframe as input and complete the full machine learning pipeline process.

Inside this function, students should:

1. Make a copy of the original dataframe.
2. Separate the data into:
 - input features X
 - target variable y
3. Use `train_test_split()` to split the original data into training and test sets.
4. Create a pipeline using `SkPipeline()`.
 - The pipeline should contain these steps in order:
 - `FunctionTransformer()` for numeric feature engineering
 - `SimpleImputer()` for missing value handling
 - `FunctionTransformer(np.log1p)` for log transformation
 - `StandardScaler()` for scaling
 - `RandomForestRegressor()` for model training
5. Fit the pipeline only on the training data using: **pipeline.fit**
6. Predict on the training data using: **pipeline.predict**
7. Predict on the test data using: **pipeline.predict**
8. Use the metric and diagnostic plot helper functions to evaluate both train and test results.
9. Return the final trained pipeline object.

Task H — Run Part-1 and Part-2

Part 1: Exploration & Visualization

The goal of this part is to show that you understand how transformations affect the data.

- **Load the Data:** Call your function to read the `Diamond_Price_Prediction.csv`.
- **Execute EDA:** Run your diagnostic functions to show the raw data distributions.
- **Manual Preprocessing:** Run the code that generates plots for the log-transformed and scaled training data so you can **visually confirm** the "before and after", before using them in the Pipeline.

Part 2: Automated Pipeline Execution

This part demonstrates the actual "production" implementation of your model.

- **Re-load the Data:** Start fresh with the raw dataset to ensure no manual changes from Part 1 leak into the pipeline.
- **Execute Pipeline:** Run the `run_pipeline_log()` function which handles the split, fit, and prediction automatically.
- **Final Evaluation:** Display the final RMSE, MAE, R2 scores, and diagnostic plots for both the training and test sets.

Strict Requirement: Separation

The assignment explicitly forbids mixing these two sections. You should follow the "Housing Pipeline" structure used in class: first, do the manual analysis to "see" the data, then run the pipeline to "solve" the problem.

5. Required Pipeline Flow

Your assignment must follow this exact order:

Order	Step	Command / Concept
1	Load original data	<code>pd.read_csv("Diamond_Price_Prediction.csv")</code>
2	Perform EDA	<code>print_diagnostics(df, "Initial EDA")</code>
3	Separate X and y	<code>X = df.drop(columns=['price']); y = df['price']</code>
4	Split data	<code>train_test_split(X, y, test_size=0.2, random_state=42)</code>
5	Build pipeline	<code>SkPipeline([...])</code>
6	Fit pipeline on train only	<code>pipeline.fit(X_train, y_train)</code>
7	Predict train and test	<code>pipeline.predict(X_train), pipeline.predict(X_test)</code>
8	Evaluate results	RMSE, MAE, R2, diagnostic plots

6. Things to Avoid

- Do not apply StandardScaler before train/test split.
- Do not call `fit_transform()` on the test data.
- Do not transform or scale the target column price.
- Do not manually transform `X_test` outside the pipeline during final model training.
- Do not split data after transformation. Split first, then fit transformations on train data.

7. Submission Checklist

- File should run from top to bottom without error.
- Initial EDA is completed before train/test split.
- Final model is trained using **`pipeline.fit(X_train, y_train)`**.
- Final test prediction is done using **`pipeline.predict(X_train)`** and **`pipeline.predict(X_test)`**.
- Metrics and diagnostic plots are shown for both train and test data, and compared / analyzed.